ChatGPT

Closing the Aggregation Gap in PIT WALL

Information-loss math

Downsampling telemetry from 50 Hz to 1 Hz is not just “coarsening.” It changes the recoverable bandwidth by a factor of 50. At 50 Hz, the Nyquist frequency is 25 Hz; at 1 Hz, it is 0.5 Hz. The Shannon sampling result says that only signals band-limited below half the sampling rate can be recovered without loss. In practice, official signal-processing guidance says downsampling must be preceded by a low-pass anti-alias filter, because otherwise frequency content above the new Nyquist limit folds into the baseband and appears as false low-frequency behavior. SciPy’s own `decimate` API makes this explicit: it downsamples only after applying an anti-aliasing filter. [1]

If your current 50 Hz → 1 Hz reduction is implemented as one-second bucket averaging, the operator is a boxcar filter followed by decimation:

$$y[k] = \frac{1}{50} \sum_{i=0}^{49} x[50k + i].$$

That averaging step has a sinc-shaped frequency response, so it is low-pass-ish, but it is not a strong anti-alias filter. A one-second boxcar only attenuates 0.5 Hz by about 3.9 dB and 0.75 Hz by about 10.5 dB; that is nowhere near enough to guarantee clean suppression of maneuver-scale energy above 0.5 Hz before 50:1 decimation. So if the pipeline is “average then subsample,” frequencies that matter for driver inputs and transient vehicle response will either be destroyed or folded into misleading low-frequency structure. Official downsampling documentation from NI, MathWorks, and SciPy all warn that filtering must be designed specifically to avoid this overlap. [2]

There is also a hard identifiability problem. If one second of one channel is represented by a single mean, a 50-sample window is being projected onto one scalar. That leaves 49 hidden degrees of freedom in that window for that channel alone, before you even consider cross-channel coupling. Two very different 50 Hz trajectories can therefore map to the same 1 Hz bucket: early braking then release versus late braking then release, a smooth corner versus a snap oversteer correction followed by recovery, or a kerb strike plus catch versus a perfectly flat segment. Once the window is aggregated, the within-second ordering is gone unless you supplied it in some other feature. This is why “kinetic hallucination” can hide inside an apparently reasonable 1 Hz trace. The loss is structural, not just statistical. [3]

The frequencies you lose are not hypothetical. U.S. DOT guidance on vehicle-dynamics validation notes that measurable lateral vehicle responses can reach about 4 Hz in light vehicles. Classic ride literature shows passenger-car body bounce resonance around 1 Hz and axle-related resonance around 10 Hz, with sport cars tending toward higher natural frequencies because handling performance pushes stiffness upward. Steering-wheel vibration measurements show dominant energy bands in the 10–60 Hz region and measured spectral content extending much higher; separate steering-vibration studies report PSD content up to 350 Hz. Kerb-strike modeling literature explicitly treats the event as impulsive tyre–suspension loading. Racing-driver studies also report that expert drivers operate with materially higher steering rate

and steering jerk, and that they apply continuous stabilizing corrections near the traction limit. All of that sits badly with a 1 Hz representation whose strict Nyquist ceiling is 0.5 Hz. [4]

That gives a clean taxonomy of what is unrecoverable versus what is merely blurred. Unrecoverable after correct 1 Hz anti-aliasing are any events whose informative content lives above 0.5 Hz: brake onset timing, trail-brake shape, throttle pick-up timing, slip peaks, catch-and-release steering, ABS chatter, wheel-hop, kerb strikes, and most cross-channel phase relationships inside a second. If you do **not** anti-alias properly, these do not become recoverable; they become aliased and misleading. What remains reasonably reconstructible are genuinely slow components such as broad speed trend, long-radius steering bias, steady throttle plateau, and slow yaw build-up across longer corners. The practical dividing line is simple: if the event's diagnostic value depends on sub-second timing, 1 Hz has already erased or corrupted it. [5]

BUILD SPEC

- Implement a telemetry audit module that computes, for every one-second window and every channel, a Welch PSD and a wavelet detail-energy decomposition, then stores `hi_freq_energy_ratio = energy(f > 0.5 Hz) / energy(all f)`.

- Treat the exact downsampling operator as a first-class object `D`. If the current pipeline uses bucket mean, encode that explicitly; if it uses sample-pick, hold-last, or FIR decimation, encode that instead.

- Produce per-corner diagnostics: `E_hi(speed)`, `E_hi(steer)`, `E_hi(brake)`, `E_hi(throttle)`, `E_hi(ax)`, `E_hi(ay)`, `E_hi(yaw_rate)`, plus a single `corner_alias_risk = max channel E_hi`.
- Python stack: `numpy`, `scipy.signal` (`welch`, `resample_poly`, `decimate`), `pywt` or `pytorch_wavelets`, `pandas`. Persist results per lap and per track-distance bin.

Forecasting at native frequency

A naïve reuse of released Granite TTM checkpoints at 50 Hz immediately exposes the scale mismatch. IBM's released Granite/TTM branches are documented around context lengths such as 512, 1024, and 1536 time points, with released forecast lengths up to 720 time points; TTM's paper also describes adaptive patching, diverse resolution sampling, and resolution-prefix tuning as the key mechanisms for handling multiple time resolutions. Reinterpreted literally at 50 Hz, those same point counts become only 10.24–30.72 seconds of context and, at the long end, 14.4 seconds of forecast. That is local-corner forecasting, not lap forecasting. So the aggregation gap is not solved by simply "feeding 50 Hz into the existing branch." The architecture itself has to become explicitly multi-rate. [6]

Direct high-rate patched TTM

The most literal solution is to keep the TTM idea but retokenize aggressively. PatchTST showed why patching is powerful for time series: it preserves local semantics while reducing token count and letting the model attend over longer history. TTM already uses patch-based processing internally and explicitly conditions on resolution. At 50 Hz, the right version is not "one sample = one token," but "a physically meaningful sub-second patch = one token." For motorsport telemetry, good patch sizes are 5, 10, 25, and 50 samples, corresponding to 100 ms, 200 ms, 500 ms, and 1 s. Short patches preserve transient control edges; longer patches capture slower corner structure. MTST's central observation is directly relevant here:

short patches are better for high-frequency localized patterns, while longer patches capture slower periodic structure. [7]

Architecturally, that means a 50 Hz TTM-class model should use multi-branch patch embeddings at several patch sizes, fuse them with a shared backbone, and decode back to 50 Hz either by predicting raw future samples or by predicting per-patch basis coefficients. The decoder should enable channel mixing, because the whole point is to stop independent-channel hallucinations. IBM's own time-series documentation states that the pretrained model is channel-independent by default and that decoder channel mixing is the mechanism for capturing cross-channel dependencies during fine-tuning. [8]

Polyphase shared-weight TTM

The most elegant native-rate answer is a polyphase decomposition. Define 50 phase streams for each channel:

$$x^{(r)}[k] = x[50k + r], \quad r \in \{0, \dots, 49\}.$$

This is standard multirate signal-processing logic: instead of averaging 50 samples into one number, you keep all 50 residue classes modulo 50 as separate low-rate streams. Each stream is observed at 1 Hz, but together the 50 phases reconstruct the original 50 Hz signal exactly. MathWorks' multirate documentation even describes obtaining polyphase components by downsampling. The critical point is that polyphase keeps **all** samples, while aggregation discards 49/50ths of them. [9]

You then run a shared-weight TTM over all phase streams with a learned phase embedding e_r and a cross-phase mixer. The shared TTM learns long-term behavior along the 1 Hz phase-time axis; the phase mixer learns within-second structure across the 50 phases. Interleaving the outputs yields a 50 Hz forecast. This effectively converts the "50 Hz native forecasting" problem into a structured family of 1 Hz problems, while preserving sub-second information exactly. It also turns released TTM horizons back into something useful: a 720-step horizon in phase-time means 720 future seconds per phase, which is more than enough for multi-lap local forecasting once the 50 phases are interleaved. [10]

Hierarchical coarse-to-fine forecasting

A second family is explicitly hierarchical. NHITS makes the key argument that multi-rate sampling plus hierarchical interpolation is a better inductive bias for long horizons than a single monolithic predictor. TimeMixer makes a similar argument in more modern terms: microscopic information lives in fine scales, macroscopic information in coarse scales, and forecasting should mix both directions. FEDformer adds frequency-domain decomposition on top, explicitly separating coarse trend from higher-frequency seasonal/detail structure. Taken together, the literature is pointing toward the same design: predict slow structure at coarse rate, fast transients at fine rate. [11]

For PIT WALL, that means a 1 Hz or 5 Hz backbone predicts slowly varying states such as corner entry speed trend, broad throttle intent, and gross steering bias, while a residual head predicts what happens **within** each second. The residual head should not predict raw samples blindly. It should predict detail coefficients in a wavelet or Laplacian pyramid, for example:

$$x_{50\text{Hz}} = U(x_{1\text{Hz}}) + d_{1\to5} + d_{5\to10} + d_{10\to25} + d_{25\to50}.$$

This is exactly the kind of coarse-to-fine decomposition the current multiscale forecasting literature rewards. [12]

Learned temporal super-resolution

You can also treat native-rate forecasting as a conditional super-resolution problem. There is now direct literature on time-series super-resolution and on super-resolution for telemetry-like coarse monitoring. The time-series SR work based on Temporal Adaptive Normalization defines the problem as reconstructing a high-resolution sequence from low-resolution measurements using a convolutional–recurrent architecture. Zoom2Net does something similar for network telemetry, reconstructing fine-grained time series from coarse-grained monitoring while enforcing operational and measurement constraints so the imputed time series remain plausible. That is almost exactly your use case, except your constraints are vehicle dynamics instead of network operations. [13]

The information-theoretic caveat matters: 1 Hz alone cannot uniquely determine a 50 Hz future, because the high-frequency content has already been removed. So super-resolution is not “recovery” in the Shannon sense; it is conditional generation under priors. In your domain that is acceptable **only** if the priors are explicit: recent 50 Hz history, track position, corner ID, adaptation flag, and a physics projector. Without those priors, a 1 Hz → 50 Hz super-resolver will just hallucinate sub-second detail. With them, it becomes a principled detail generator that can be audited. [14]

Continuous-time coefficient forecasting

The broader irregular-time literature offers another route: do not forecast points at all; forecast a continuous trajectory representation. Interpolation-Prediction Networks insert a semi-parametric interpolation stage before prediction. Latent ODEs and Neural CDEs model latent dynamics in continuous time and are designed precisely for irregularly sampled multivariate sequences. For PIT WALL, the right translation is to have the coarse model predict spline knots, basis coefficients, or latent continuous-time states, and then sample the trajectory at 50 Hz. This is especially attractive if some channels are naturally asynchronous or if you want distance-based and time-based views to coexist. [15]

Long-context backbones for full native-rate sequences

If you insist on forecasting entire lap-scale 50 Hz sequences directly, then the backbone itself has to change. S4 and Mamba exist because quadratic-attention Transformers do not scale gracefully on long sequences, while structured state-space models scale linearly and retain long-range memory. For raw 50 Hz telemetry across one or more laps, a linear-time SSM backbone with TTM-style patch embeddings is the cleanest native-rate route. This is no longer “plain Granite TTM,” but it is absolutely within the same foundation-model design space. [16]

BUILD SPEC

- Implement a **polyphase-multiresolution hybrid**:
- exact 50-phase decomposition per channel;
- shared-weight TTM branch over phase-time with learnable phase embeddings;
- cross-phase mixer using 1D depthwise-separable convolution or lightweight attention across the 50 phases;

- coarse 1 Hz backbone plus wavelet-detail residual heads at 5, 10, 25, and 50 Hz.
- Use IBM Granite TTM weights as the shared low-rate backbone, with a new resolution token for `20ms` data and decoder channel mixing enabled. [17]
- Add a temporal super-resolution side head conditioned on coarse forecast, recent raw 50 Hz context, and track-distance embedding. Train it with aggregate-consistency loss, detail loss, and physics-projection loss.
- Python stack: `torch`, `transformers` / `granite-tsfm`, `einops`, `pywt` or `pytorch_wavelets`, `scipy.signal`, optional `mamba-ssm` or an S4 implementation for full native-rate experiments.

Native-resolution physics projection

Yes: the physics-projection layer can operate at 50 Hz even if the forecaster runs at 1 Hz, and the gradient can still flow through an upsample → project → downsample chain. In fact, this is one of the cleanest ways to answer the aggregation critique, because it turns low-rate forecasting into a **feasible-lift** problem: a 1 Hz prediction is accepted only insofar as it admits a physically valid 50 Hz realization. Differentiable convex optimization layers exist precisely for this use case. The CVXPYLayers paper formalizes disciplined parametrized convex programs, shows how they can be converted to an affine-solver-affine form, and differentiates analytically through the solution map; the library's Python implementation exposes this directly for PyTorch, JAX, and MLX. OptNet established the same principle earlier for differentiable QP layers. [18]

Write the 1 Hz forecast as $y \in \mathbb{R}^{T \times C}$. Let U_θ be a differentiable upsampler that maps this into a 50 Hz initial guess $\bar{z} \in \mathbb{R}^{50T \times d}$, where d includes the high-rate channels the projector needs. Then solve

$$z^\star = \arg\min_z \ \|W_m(z - \bar{z})\|_2^2 + \lambda_v \|\Delta z\|_2^2 + \lambda_j \|\Delta^2 z\|_2^2 + \lambda_a \|Dz - y\|_2^2$$

subject to high-rate vehicle constraints, where D is the exact aggregation operator used by preprocessing, Δ is a first-difference operator, and Δ^2 penalizes jerk-like roughness. If preprocessing used bucket means, D is block averaging; if preprocessing used point samples or polyphase picks, D must mirror that exactly. The projected low-rate output is either $\hat{y} = Dz^\star$ or the full projected sequence $z^\star$ itself. Because U_θ and D are smooth or linear and the optimization layer is differentiable, backpropagation flows end to end. [19]

At 50 Hz, the projector should enforce the constraints that actually matter at the hidden timescale. The minimal convex core is a lifted kinematic/dynamic model with high-rate state variables, actuator-rate constraints, and a second-order-cone friction constraint such as

$$\left\| \begin{bmatrix} a_x/(\mu_x g) \\ a_y/(\mu_y g) \end{bmatrix} \right\|_2 \le 1.$$

You then add steering-rate bounds, yaw/heading integration, speed nonnegativity, and channel-consistency equations. The downsampled forecast is now valid only if there exists a 50 Hz trajectory satisfying those conditions. That is a much stronger statement than "the 1 Hz points look plausible." It says the 1 Hz forecast has a physically valid high-rate witness. [20]

The part that needs care is nonconvexity. Exact bicycle dynamics with realistic tire force maps, slip-angle coupling, or strict throttle–brake complementarity are not convex. If you want a clean CVXPYLayer, you keep

the projection core convex and use linearizations, cone relaxations, or soft penalties. If you want the richer frontier, you run sequential convex programming: linearize around the previous iterate, solve a QP/SOCP, update, and unroll a fixed number of iterations. Differentiation then happens through the unrolled solver or through the final KKT system for each convex subproblem. This is still compatible with a differentiable pipeline; it is just not a single closed-form convex layer. [21]

One subtle but important point: downsampling after projection should stay differentiable. Means, FIR low-pass operations, spline evaluations, and polyphase selects are fine. Hard maxima and minima are not ideal if you want smooth gradients; replace them with softmax or log-sum-exp approximations when you need "peak brake," "peak lateral g," or "max steering rate" inside a differentiable loss. SciPy and MathWorks documentation emphasize exactly these linear filtering and resampling operations in multirate pipelines. [22]

BUILD SPEC

- Define `D` and `U_theta` explicitly:
- `D`: the exact current 50 Hz → 1 Hz preprocessing operator;
- `U_theta`: a differentiable upsampler implemented as cubic spline, FIR transposed convolution, or a learned 1D interpolation network.

- Build a 50 Hz projection layer over variables `z_t = [vx, vy, yaw_rate, heading, steer, throttle, brake, ax, ay, speed]` with objective `tracking + smoothness + jerk + aggregate-consistency`.

- Implement the projector as:

- convex SOCP/QP core in `cvxpy` + `cvxpylayers`,
- optional unrolled sequential-convex outer loop for richer tire/combined-slip dynamics,
- smooth overlap penalty for throttle/brake exclusivity when the COA flag says overlap is disallowed.
- Python stack: `torch`, `cvxpy`, `cvxpylayers`, `diffcp`, `scipy.interpolate` or a Torch spline implementation. Use averages/FIR filters in `D`; use softmax/log-sum-exp surrogates instead of hard window extrema.

Multi-rate and irregular time-series literature

The strongest high-confidence takeaway from the literature is that state-of-the-art forecasting is not moving toward a single universal fixed-rate predictor. It is moving toward explicit handling of scale: patches, hierarchies, frequency decompositions, and continuous-time representations. PatchTST showed that treating subseries patches as tokens retains local semantics and makes long-history forecasting tractable. TTM extends that logic with adaptive patching and explicit resolution conditioning. MTST then pushes the argument further by building multiple branches for different patch sizes because distinct resolutions carry distinct temporal patterns. This family maps almost perfectly to racing telemetry, where slow corner evolution and fast stabilization transients inhabit different scales. [7]

A second family is coarse-to-fine hierarchy. NHITS combines multi-rate input sampling and hierarchical interpolation, explicitly decomposing forecasts into components with different frequencies and scales.

TimeMixer makes the same intuition more explicit: microscopic information belongs to fine scales and macroscopic information to coarse scales, so the model should mix both directions. FEDformer pushes scale separation into the frequency domain, using decomposition and sparse Fourier or wavelet modes to allocate capacity to detail where detail actually lives. For your problem, this literature says the right answer is not “pick one sample rate,” but “forecast a slow backbone and a stack of finer-scale corrections.” [23]

A third family handles irregular or asynchronous time directly. Interpolation-Prediction Networks insert a learned interpolation layer before the predictor. Latent ODEs and Neural CDEs put the hidden state in continuous time instead of tying it to fixed ticks. RAINDROP and Multi-Time Attention Networks extend this to irregular multivariate sensors with explicit time encodings and cross-sensor structure. This matters because motorsport data is often only *nominally* regular: different sensors may have different effective latencies, packet drops happen, derived channels are computed at different rates, and track-distance and wall-clock are both useful coordinate systems. Continuous-time modeling lets you resolve these without flattening everything into a single rigid clock. [24]

A fourth family is long-context efficient sequence modeling. S4 and Mamba exist because long sequences are expensive under quadratic attention and because linear-time state-space models can retain far longer context. At 50 Hz, even one minute is 3,000 points per channel; multi-lap context is tens of thousands. If you want native-rate full-lap forecasting instead of corner-local forecasting, this literature is the mechanism that keeps the compute linear enough to be realistic while still preserving long-term dependencies. [16]

For PIT WALL, the literature therefore maps into a very clear stack: patching for token efficiency, hierarchy for scale separation, continuous-time modeling for asynchronous or distance-based representation, and state-space backbones where full-rate sequence length becomes the bottleneck. The field is not telling you to choose one. It is telling you to combine them. That is an inference from the convergence of the representative primary papers above, not a claim from any single paper. [25]

BUILD SPEC

- Build the model stack as four interoperable layers:
- **patch layer** for 100 ms / 200 ms / 500 ms / 1 s blocks,
- **hierarchy layer** for 1, 5, 10, 25, 50 Hz forecasts,
- **continuous-time layer** using splines or Neural CDE-style interpolation for irregularity and distance/time dual indexing,
- **long-context backbone** using TTM first and an S4/Mamba branch for full-lap native-rate experiments.
- Encode time in both **elapsed time** and **track distance**; use distance as the alignment axis for corner-local modules and time for dynamics.

- Treat each forecast as a tuple `(coarse trajectory, multiscale detail coefficients, continuous-time interpolant, projected 50 Hz witness)`.

- Python stack: `torch`, `granite-tsfm`, `torchcde` or equivalent, `pywt`, optional `mamba-ssm` / S4 implementation.

Detection from aggregation discrepancy

Yes: the discrepancy between a 1 Hz aggregate and its hidden 50 Hz content can itself become a first-class signal. In fact, the anomaly-detection literature argues strongly that multiresolution representations surface abnormalities that single-scale models miss. MADE detects anomalies by combining time and frequency context and explicitly separates fast- and slow-scale anomalies with wavelets. RAMED uses multiresolution ensemble decoding. MS2D-Net treats multiresolution downsampling patterns as self-supervision. MSCRED uses multi-scale signature matrices to capture system status at different resolutions. The common result is that anomalies often live in the *relationship between scales*, not in either scale taken alone. [26]

For PIT WALL, the most direct detector is a **hidden-energy score**. Given a historical 50 Hz one-second window w_k, compute

$$R_{\text{hidden}}(k) = \frac{\sum_{f > 0.5\ \text{Hz}} |X_k(f)|^2}{\sum_{f \geq 0} |X_k(f)|^2}.$$

That score is literally the fraction of energy that 1 Hz cannot faithfully represent. You can compute it per channel and per corner, then train a predictor that estimates R_{hidden} from aggregate-only features. At inference time, windows with high predicted hidden energy become “sub-second blind-spot” alerts even if the 1 Hz forecast itself looks clean. This is not forecasting; it is uncertainty surfacing caused by aggregation. [27]

The second detector is the **feasible-lift gap**. Given a 1 Hz bucket y_k, solve

$$\rho_k = \min_{z \in \mathcal{F}} \|Dz - y_k\|_2^2,$$

where $\mathcal{F}$ is the set of physically feasible 50 Hz trajectories under your projector constraints. If ρ_k is large, there is no nearby 50 Hz physically valid realization of that bucket. That means the bucket is suspicious **even before** you ask a forecaster to extrapolate it. This is the right mathematical detector for hidden kinetic hallucinations, because it asks whether the aggregate has any physically valid high-rate witness. [21]

The third detector is **subgrid ambiguity spread**. Define a physically important latent quantity $g(z)$, such as peak combined-slip utilization, peak steering-rate correction, peak brake pressure, or minimum steering consistent with observed lateral acceleration. Then solve a pair of robust optimization problems over the feasible high-rate fiber:

$$g_{\min}(y_k) = \min_{z \in \mathcal{F},\ Dz = y_k} g(z), \qquad g_{\max}(y_k) = \max_{z \in \mathcal{F},\ Dz = y_k} g(z).$$

If $g_{\max} - g_{\min}$ is wide, the 1 Hz bucket is underdetermined in exactly the region you care about. That interval width is a direct “hallucination hiding place” score. The wider the interval, the more room there is for dangerous hidden behavior under the same 1 Hz aggregate. This is particularly useful for corner ranking, because corners with the largest ambiguity intervals are the corners where a judge’s criticism is strongest. [28]

The fourth detector is a **cross-scale reconstruction residual**. Train a multiresolution super-resolver or reconstruction model on normal 50 Hz history, then compare its internally consistent 50 Hz reconstruction

against what is implied by the 1 Hz bucket and by the physics projector. Multiscale anomaly papers use exactly these residual and reconstruction inconsistencies as anomaly evidence. In PIT WALL, the same idea becomes an “aggregation discrepancy monitor”: high reconstruction residual, large projection correction cost, or high ensemble variance across multiple SR samples all indicate that the bucket is hiding structure the coarse representation cannot pin down. [29]

The most useful operational framing is not a binary anomaly flag. It is a **risk heatmap over track distance** with three channels: hidden energy, feasible-lift gap, and ambiguity spread. That lets PIT WALL say, in plain English, “Turn 4 exit is low-rate-safe,” “Turn 7 brake zone is aggregation-unsafe,” or “Turn 10 kerb event likely contains hidden sub-second dynamics.” That is far harder to attack than a generic warning banner. [30]

BUILD SPEC

- For every one-second window, compute and persist:
- `hidden_energy_ratio` by PSD or wavelet detail energy,
- `feasible_lift_gap` from the 50 Hz projector,
- `ambiguity_spread` for at least three critical latent quantities: peak combined-slip, peak steering-rate correction, peak brake/throttle overlap risk.
- Train a lightweight risk model on historical paired data:
- inputs: aggregate-only features (`mean` , `std` , `max-min` , total variation, coarse derivatives, corner ID, distance bin),
- targets: `hidden_energy_ratio` , `feasible_lift_gap` , `ambiguity_spread` .
- Expose corner-level outputs:
- `aggregation_safe` ,
- `aggregation_unsafe` ,
- `aggregation_ambiguous` , each with numeric confidence and explanatory factors.
- Python stack: `numpy` , `scipy` , `pywt` , `cvxpy` / `cvxpylayers` , and a small tabular learner such as `xgboost` or a compact `torch` MLP.

Highest-leverage approach

The single highest-leverage approach is **polyphase-native forecasting with a 50 Hz feasible-lift physics projector**.

It has the best leverage because it solves the root problem instead of papering over it. Polyphase decomposition keeps **all** 50 Hz samples exactly, but rearranges them into 50 structured 1 Hz phase streams. That means you do not pay the catastrophic information loss of aggregation, and you do not force the backbone to operate on 50× longer scalar sequences. It also means Granite TTM’s released context and horizon lengths become useful again in phase-time rather than collapsing to a few seconds at 50 Hz. Then, after forecasting, a differentiable 50 Hz projector asks the only question that really matters: does this coarse prediction admit a physically valid high-rate witness? If not, the model corrects or rejects it. That combination directly attacks the judge’s strongest line of criticism. [31]

In concrete form, the architecture is:

$$50\ \text{Hz telemetry} \rightarrow \text{polyphase decomposition} \rightarrow \text{shared-weight TTM} + \text{phase embeddings} + \text{cross-phase mixer} \rightarrow$$

That gives you exact retention of native-rate information, long horizon in a TTM-like representation, explicit cross-phase modeling of within-second structure, and a mathematically auditable safeguard against hidden kinetic hallucinations. Nothing else buys as much signal fidelity, modeling efficiency, and defensibility in one move. [32]

Open questions and limitations

I found strong primary support for the sampling math, multirate forecasting architectures, differentiable optimization layers, and the frequency content of vehicle lateral/ride/steering vibration signals. I did **not** find equally strong motorsport-specific primary spectral studies for every single raw telemetry channel, especially brake and throttle PSDs in real race cars. Where I describe those channels as sub-second-critical, that conclusion is high-confidence and physically well grounded, but some of the channel-by-channel frequency characterization is inferred from adjacent vehicle-dynamics and steering-transient literature rather than from a single race-telemetry PSD paper. The other material limitation is mathematical: a truly rich 50 Hz vehicle model with exact tire-force nonlinearities and strict complementarity constraints is not globally convex, so the cleanest differentiable implementation is an unrolled sequential-convex or differentiable nonlinear solver rather than one pure QP layer. [33]

---

[1] [5] [14] [27] https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf
https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf

[2] [3] https://www.ni.com/en/shop/data-acquisition/measurement-fundamentals/analog-fundamentals/anti-aliasing-filters-and-their-usage-explained.html?srsltid=AfmBOoq0pqOND8pH50uHrfKeHkKlMKW1_TtpTGTiFvgVOE62Z1y-ig9H
https://www.ni.com/en/shop/data-acquisition/measurement-fundamentals/analog-fundamentals/anti-aliasing-filters-and-their-usage-explained.html?srsltid=AfmBOoq0pqOND8pH50uHrfKeHkKlMKW1_TtpTGTiFvgVOE62Z1y-ig9H

[4] [33] https://rosap.ntl.bts.gov/view/dot/57509/dot_57509_DS1.pdf
https://rosap.ntl.bts.gov/view/dot/57509/dot_57509_DS1.pdf

[6] [10] https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2/blob/5b53f7d55d58e6af41957533437d6a1c86c59b61/README.md
https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2/blob/5b53f7d55d58e6af41957533437d6a1c86c59b61/README.md

[7] [25] arxiv.org
https://arxiv.org/pdf/2211.14730

[8] [17] https://www.ibm.com/granite/docs/fine-tune/time-series
https://www.ibm.com/granite/docs/fine-tune/time-series

[9] [31] [32] https://www.mathworks.com/help/signal/multirate-signal-processing.html
https://www.mathworks.com/help/signal/multirate-signal-processing.html

[11] [12] [23] NHITS: Neural Hierarchical Interpolation for Time Series Forecasting
https://ojs.aaai.org/index.php/AAAI/article/view/25854/25626

13 https://openreview.net/pdf?id=H1lWCSQdi7
https://openreview.net/pdf?id=H1lWCSQdi7

15 24 https://openreview.net/forum?id=r1efr3C9Ym
https://openreview.net/forum?id=r1efr3C9Ym

16 https://arxiv.org/abs/2111.00396
https://arxiv.org/abs/2111.00396

18 19 20 21 28 https://stanford.edu/~boyd/papers/pdf/diff_cvxpy.pdf
https://stanford.edu/~boyd/papers/pdf/diff_cvxpy.pdf

22 https://docs.scipy.org/doc/scipy-1.16.1/reference/generated/scipy.signal.decimate.html
https://docs.scipy.org/doc/scipy-1.16.1/reference/generated/scipy.signal.decimate.html

26 30 https://www.sciencedirect.com/science/article/abs/pii/S0022169424010631
https://www.sciencedirect.com/science/article/abs/pii/S0022169424010631

29 https://ojs.aaai.org/index.php/AAAI/article/view/17152
https://ojs.aaai.org/index.php/AAAI/article/view/17152